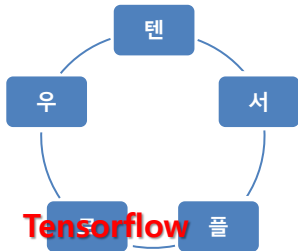




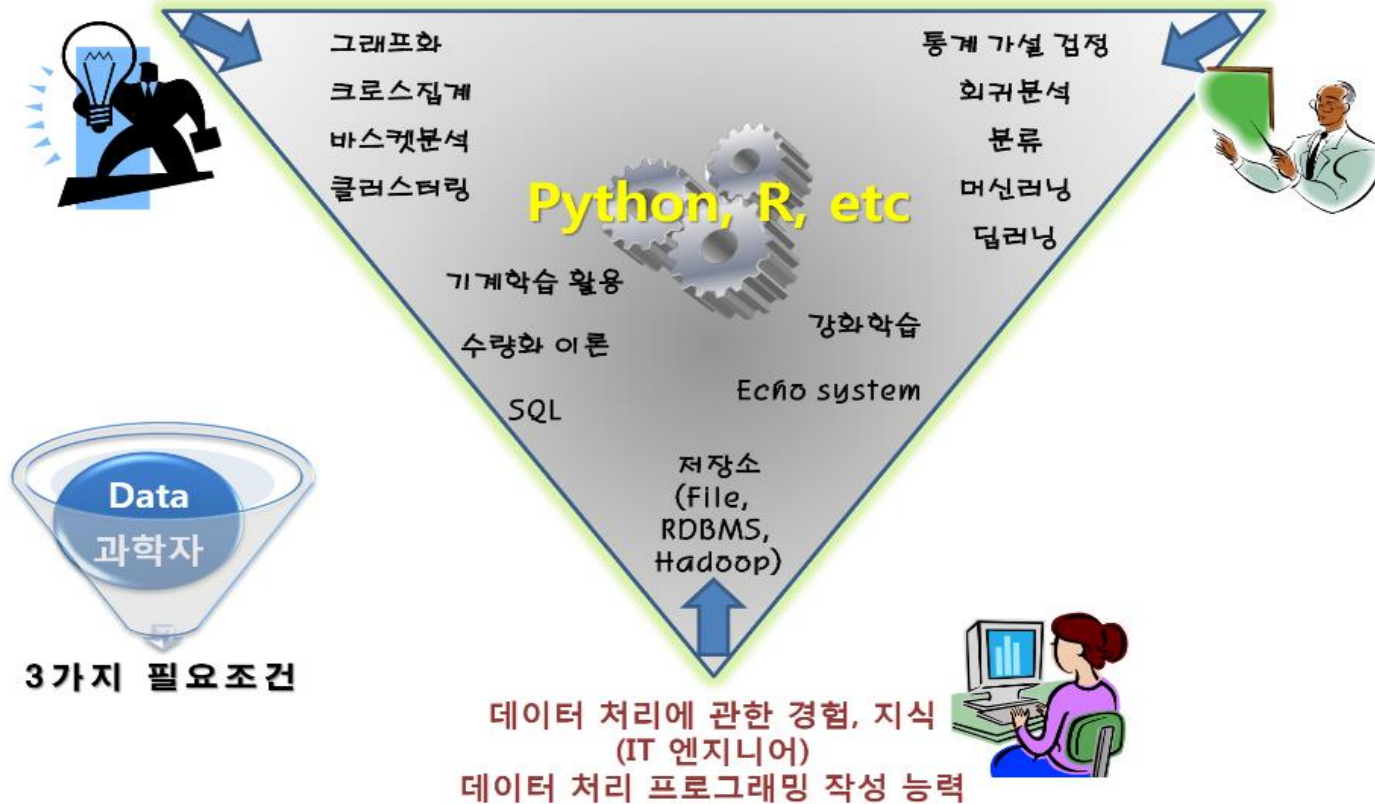
TensorFlow



pyk

비즈니스에 관한 경험, 지식
(기획, 영업)
고객의 마음을 알아내는 능력

설계기법에 관한 경험, 지식
(학자, 연구원)
통계해석 능력



데이터 분석가 :

데이터를 이용해 질문에 답하고 결과를 전달하여 비즈니스 의사 결정에 도움을 줌으로써 업무에 가치를 제공한다.
데이터 정리, 분석 수행 및 데이터 시각화 생성도 이들이 하는 일이다.

데이터 과학자 :

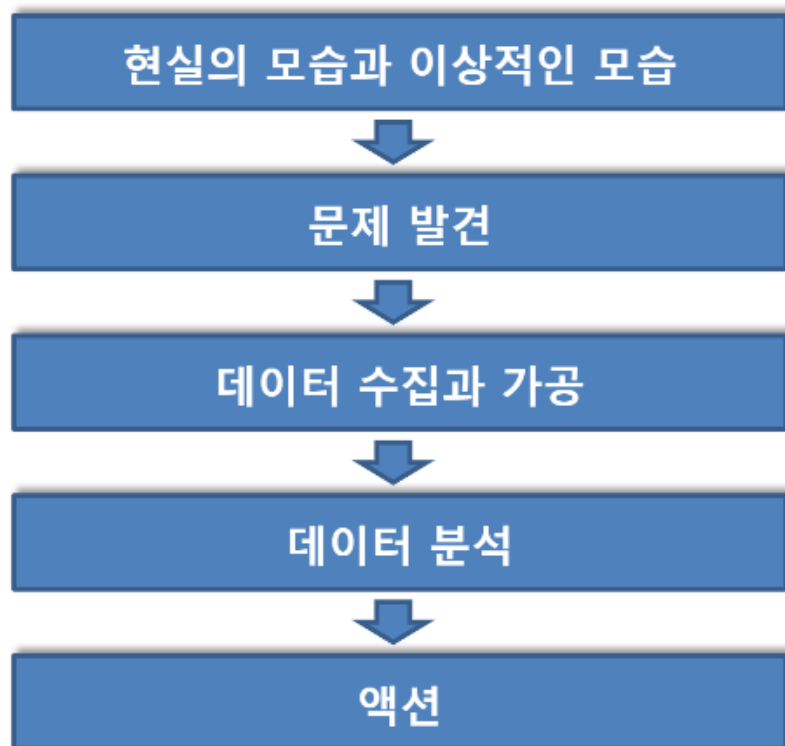
통계 및 기계 학습 모델 구축에 대한 전문 지식을 적용하여 결과를 예측하고 주요 비즈니스 질문에 답변하는 작업을 한다.
분석가 처럼 데이터를 정리, 분석 및 시각화 할 수 있으며 이런 기술에 대한 전문 지식을 갖추고 기계 학습 모델을 교육하고 최적화 할 수 있다.

데이터 엔지니어 :

데이터 과학자와 분석가가 데이터를 사용하여 작업을 수행 할 수 있도록 시스템을 구축하고 최적화한다.
모든 데이터가 올바르게 수신, 변환, 저장 및 다른 사용자가 액세스 할 수 있도록한다.

Business에서 데이터 분석의 흐름

데이터 분석이란 현재 상황으로부터 이상적인 모습에 빨리 도달하기 위해 문제를 추출하는 것에 주안점을 두고 아래와 같은 순서대로 실시하면 효과적이다.



비즈니스에서 데이터 분석은 비즈니스에서 발생한 여러 가지 질문을 통계해석이나 기계학습, 데이터 마이닝 등의 각종 방법론을 구사하여 해결하는 것이 목적이다.

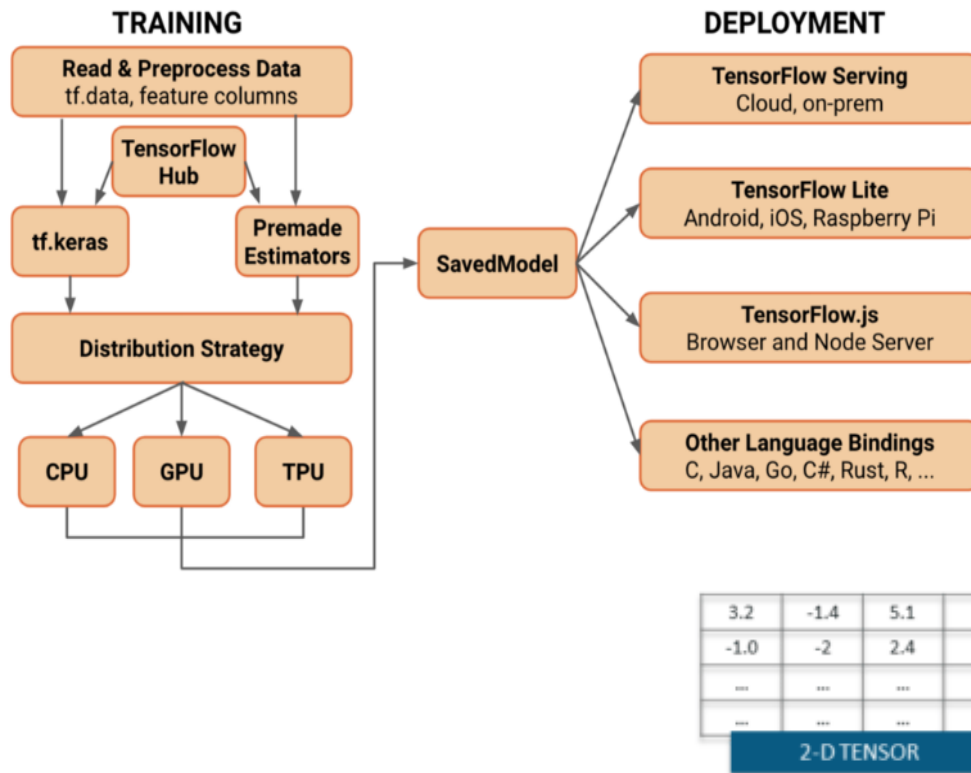
이 때에 고도의 복잡한 모델을 통해 분석한 결과가 반드시 높은 가치가 있다고 볼 수는 없다. 간단하게 분할표를 만들어 단시간에 분석결과를 얻어 내는 것이 더 큰 가치가 있을 수도 있다.

주어진 문제에 대해 맞는 분석기법을 설계하고 실행하는 것이 가장 중요하며 이 것을 간과하면 데이터 분석결과를 투자에 비해 가치가 떨어질 가능성이 있다는 점을 잊지 말자!

Tensorflow

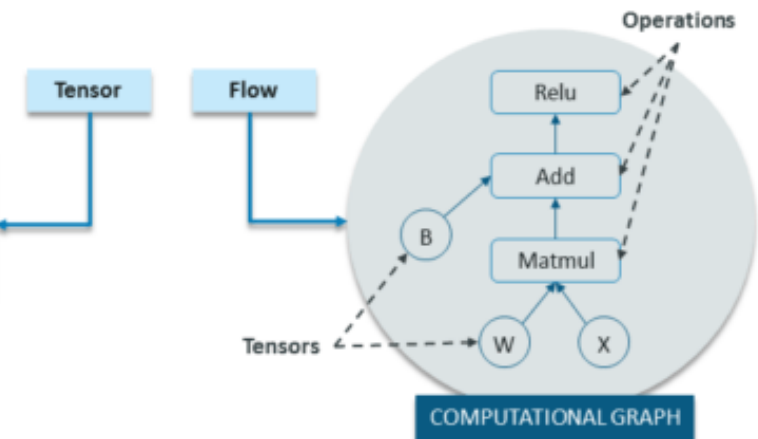
- 구글이 공개한 Deep Learning Framework
- Apache 오픈 소스 라이브러리
- C/C++로 개발된 엔진 위에 Python을 지원하는 구조를 갖고 있다.
- 딥러닝 프로그램을 쉽게 구현할 수 있도록 다양한 기능을 제공하는 "머신러닝 라이브러리"다.
- Tensorflow core는 완벽한 프로그래밍 제어 기능을 제공한다.
머신러닝 연구자가 모델을 제어하고자 할 경우에 사용하면 효과적이다.

간단하고 개념적인 다이어그램을 통해 텐서플로우 2.0의 새로운 아키텍처



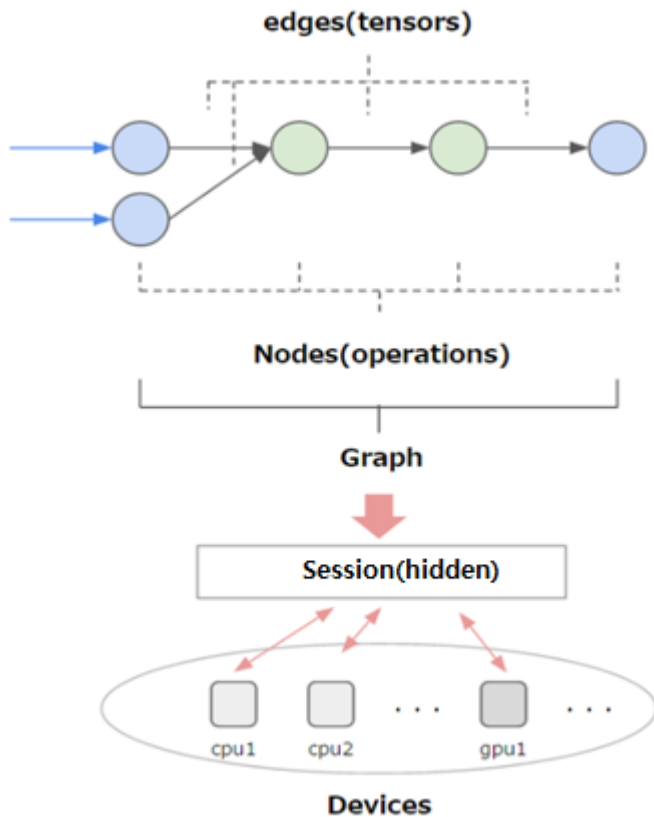
특징

- 데이터 플로우 그래프를 통한 풍부한 표현력
- 코드 수정 없이 CPU/GPU 모드로 동작.
- GPU에 최적화.
- 아이디어 테스트에서 서비스 단계까지 이용 가능
- 계산 구조와 목표 함수만 정의하면 자동으로 미분 처리
- Python/C++를 지원하며, SWIG를 통해 다양한 언어 지원
- 분산병렬 컴퓨팅 환경을 지원
- '텐서보드'라는 시각화 도구 제공.
- 딥러닝 라이브러리로 Keras를 채택(단일화)



Tensorflow 구조

텐서플로우는 edges와 nodes 구성.
구조화된 Graph로 프로그램이 처리.



```
import tensorflow as tf
```

```
a = tf.constant(1)  
print(a)
```

2 feed data and run graph (operation)
sess.run (op)

1 Build graph using
TensorFlow operations



3 update variables
in the graph
(and return values)

Tensorflow : 처리 순서

텐서플로우는 빌딩구조와 실행구조(session)가 합쳐진 형태로 처리된다.

* 처리 순서

1. tensorflow 모듈을 가져 와서 tf 호출
2. x라는 상수 값을 만들고 숫자 값 1 기억.
3. y라는 변수를 만들고 방정식 $x + 5$ 로 정의
4. 그래프 모델 실행
5. 변수 y 만 실행하고 현재 값을 출력

```
import tensorflow as tf

x = tf.constant(1)
y = tf.Variable(x + 5)
print(y)
```

Tensorflow 2.x는

Session 객체가 내부적으로 수행됨. 명시적인 선언이 없다.

Placeholder 객체를 사용하지 않는다.

Variable() 객체에 대한 초기화 코드를 기술하지 않는다.

...

Tensor 용어

텐서는 0 차원 부터 n 차원까지를 가지는 대표적인 데이터 클래스

0차 텐서 : 벡터가 없기 때문에 스칼라임
1차 텐서 : 벡터가 하나 있기 때문에 벡터임
2차 텐서 : 벡터가 두 개 일렬로 있기 때문에 행렬
N차 텐서 : n-1 차 텐서가 일렬로 구성됨

```
import tensorflow as tf
import os
```

```
# SSE 및 AVX 등의 경고는 소스를 빌드 하면 없어지지만, 명시적으로 경고 없애기
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'
```

```
print("즉시 실행 모드: ", tf.executing_eagerly())
print("GPU ", "사용 가능" if tf.test.is_gpu_available() else "사용 불가능")
```

```
# 상수 정의(생성) - 상수 텐서를 생성한다.
```

```
print(tf.constant(1))      # tf.Tensor(1, shape=(), dtype=int32)
print(tf.constant([1]))    # tf.Tensor([1], shape=(1,), dtype=int32)
print(tf.constant([[1]]))  # tf.Tensor([[1]], shape=(1, 1), dtype=int32)
```

Tensor에 대한 data type

data type	Python type	Description
DT_FLOAT	tf.float32	32 bits floating point.
DT_DOUBLE	tf.float64	64 bits floating point.
DT_INT8	tf.int8	8 bits signed integer.
DT_INT16	tf.int16	16 bits signed integer.
DT_INT32	tf.int32	32 bits signed integer.
DT_INT64	tf.int64	64 bits signed integer.
DT_UINT8	tf.uint8	8 bits unsigned integer.
DT_STRING	tf.string	Variable length byte arrays. Each element of a Tensor is a byte array.
DT_BOOL	tf.bool	Boolean.
DT_COMPLEX64	tf.complex64	Complex number made of two 32 bits floating points: real and imaginary parts.
DT_COMPLEX128	tf.complex128	Complex number made of two 64 bits floating points: real and imaginary parts.
DT_QINT8	tf.qint8	8 bits signed integer used in quantized Ops.
DT_QINT32	tf.qint32	32 bits signed integer used in quantized Ops.
DT_QUINT8	tf.quint8	8 bits unsigned integer used in quantized Ops.

변수

`w = tf.Variable(<initial-value>, name=<optional-name>) # Create a variable.`

변수는 생성되는 순간에 데이터의 크기와 타입이 결정된다.

변수를 만들면 초기 값으로 Tensor를 Variable() 생성자에 전달한다.

TensorFlow는 상수 또는 임의 값에서 초기화에 자주 사용되는 텐서를 생성하는 연산 모음을 제공한다.

이러한 모든 작업에는 텐서의 모양(shape)을 지정해야 한다. 그 모양은 자동으로 변수의 모양이 된다.

변수는 일반적으로 고정된 모양을 갖지만 TensorFlow는 변수를 변경하는 고급 메커니즘을 제공한다.

변수 정의는 tf.Variable()로 정의하고 출력해보면 Tensor 클래스의 값으로 나옴

```
import tensorflow as tf

x = tf.constant(35, name='x')
y = tf.Variable(x + 5, name='y')

print(y)
```

```
Tensor("y_2/read:0", shape=(), dtype=int32)
```

Tensorflow에서 텐서를 기억하기 위한 메모리 확보 방법

- constant : tensor(일반적인 상수 값)를 직접 기억
- Variable : tensor가 저장된 주소를 참조

Numpy vs. Tensor

Numpy 는 큰 다차원 배열과 행렬을 지원하고 이러한 요소에서 작동하는 수학 함수 모음을 제공. 이는 Fortran과 같은 다른 언어로 구현 된 잘 알려진 패키지를 사용하여 효율적인 계산을 수행하여 사용자에게 Python의 표현 성과 MATLAB 또는 Fortran과 유사한 성능을 제공한다.

Tensor는 다차원 배열로 NumPy의 ndarray 객체와 비슷하며, tf.Tensor 객체는 데이터 타입과 크기를 가지고 있다. 또한 tf.Tensor는 GPU 같은 가속기 메모리에 상주할 수 있다. 텐서플로는 텐서를 생성하고 이용하는 풍부한 연산 라이브러리(tf.add, tf.matmul 등)를 제공한다. 이런 연산은 자동으로 텐서를 파이썬 native type으로 변환한다.

넘파이 호환성 : 텐서와 넘파이 배열 사이의 변환은 간단하다.

텐서플로 연산은 자동으로 넘파이 배열을 텐서로 변환한다.

넘파이 연산도 자동으로 텐서를 넘파이 배열로 변환한다.

텐서는 .numpy() 메서드(method)를 호출하여 넘파이 배열로 변환할 수 있다.

가능한 경우, tf.Tensor와 배열은 메모리 표현을 공유하기 때문에 이러한 변환은 일반적으로 간단(저렴)하다. 그러나 tf.Tensor는 GPU 메모리에 저장될 수 있고, 넘파이 배열은 항상 호스트 메모리에 저장되므로, 이러한 변환이 항상 가능한 것은 아니다. 따라서 GPU에서 호스트 메모리로 복사가 필요하다.

```
import numpy as np
ndarray = np.ones([3, 3])
print("텐서플로 연산은 자동으로 넘파이 배열을 텐서로 변환!")
tensor = tf.multiply(ndarray, 42)
print(tensor)

print("넘파이 연산은 자동으로 텐서를 넘파이 배열로 변환~~")
print(np.add(tensor, 1))
print(".numpy() 메서드는 텐서를 넘파이 배열로 변환")
print(tensor.numpy())
```

텐서플로 연산은 자동으로 넘파이 배열을 텐서로 변환!

```
tf.Tensor(
[[42. 42. 42.]
 [42. 42. 42.]
 [42. 42. 42.]], shape=(3, 3), dtype=float64)
넘파이 연산은 자동적으로 텐서를 넘파이 배열로 변환~~.
[[43. 43. 43.]
 [43. 43. 43.]
 [43. 43. 43.]]
.numpy() 메서드는 텐서를 넘파이 배열로 변환합니다.
[[42. 42. 42.]
 [42. 42. 42.]
 [42. 42. 42.]]
```

Tensorflow의 연산자와 주요 함수

cond() 삼항 연산

case() 조건 체크 후 연산 처리

산술 / 관계 / 논리 연산

rank : tensor에 대한 rank 즉 차원을 조회하는 함수

shape : tensor에 대한 shape를 조회하는 함수

size : tensor에 대한 shape를 조회하는 함수

zeros/zeros_like : numpy와 tensorflow는 array 정의 기준이 다르고 실제 값을 확인할 경우 실행환경에 차이가 있음

ones/ones_like : 원소가 1을 가지는 tensor를 생성

random_normal : 2행 3열의 normal 분포 텐서를 생성

random_uniform : uniform 분포 텐서를 반복해서 생성시 동일한 값을 유지할 경우 사용

linspace : tensor를 할 때 시작점과 종료점 그리고 개수를 정해서 만들어 냄. 마지막도 범위에 포함

range : tensor를 할 때 시작점과 종료점 그리고 차이를 정해서 만들어 냄. 마지막도 범위에 포함되지 않음.

...

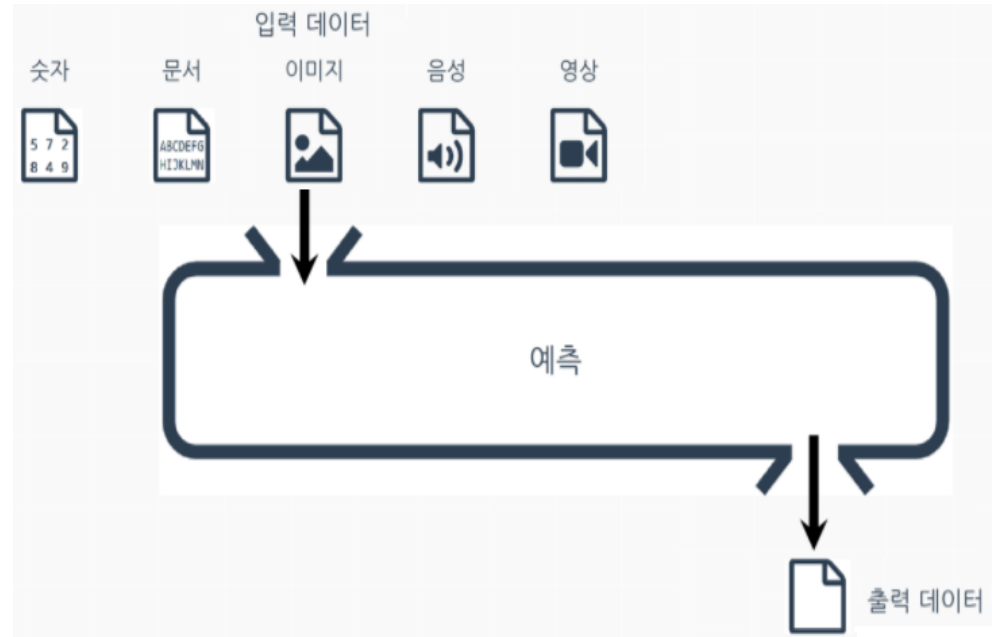
Keras 모델을 이용한 분석 방법

- Keras
- 회귀분석
- 로지스틱 회귀
- Softmax
- CNN
- RNN
- 기타



* 데이터 분석 *

데이터 분석에서 말하는 예측이라는 용어는 시간상으로 미래의 의미는 포함하지 않는다. 시계열 분석에서는 시간상으로 미래의 데이터를 예측하는 경우가 있는데 이 경우에는 미래 예측(forecasting)이라는 용어를 사용한다.



* Machine Learning(기계학습) *

알고리즘을 통해 기계에게 학습을 시킨 후에 새로운 자료가 입력된 경우 해당 자료의 결과를 분석하여 예측결과를 제공해 준다. 예를 들어 검색어 자동 완성, 악성코드 감지, 자료 인식 등의 다양한 예측을 필요로 하는 분야에서 사용될 수 있다.

- 일상에서 접하는 Machine Learning
 - 상품의 추천/ 스팸 메일 분류/ 쿠폰발급/ 대출심사 / 부동산 가격 예측 등
- 기업에서 적용하는 Machine Learning
 - Business decision / Productivity 증대/ Disease detection / Anomaly detection / Forecasting weather

** Machine Learning(기계학습) **

지도학습(Supervised Learning)은 훈련 데이터로 부터 예측/추정, 분류 함수를 만들어 내는 기계학습 방법이다. 이는 사전에 입출력에 대한 사전정보를 어느 정도 갖고 있는 상태에서 입력된 자료에 대한 모델을 만들고, 이를 통해 추정 및 예측을 할 수 있다. 독립변수들과 종속변수를 가지며, 사람의 지도 하에 독립변수와 종속변수 간에 관계를 일반화 하는 학습을 하게 된다. Y값을 가지고 있으니 학습이 끝나고 나면 결과가 나온다. 과거의 데이터를 가지고 학습을 시키고 나면, 학습 모델에 새로운 데이터에 적용해서 예측/추정, 분류 등의 작업을 수행한다.

: Regression, Support Vector Machine ...

비지도학습(Unsupervised Learning)은 관찰한 데이터로 부터 숨겨진 패턴/규칙을 탐색, 찾아내는 기계학습 방법이다. 이는 최종적인 정보가 없는 상태에서 기계 스스로가 정해진 패턴에 의해 분석결과를 만들어 내게 된다.

비지도학습에서는 종속변수(y) 값이 없고, 사람의 개입도 없다. 그냥 Input 변수 ($x_1, x_2, \dots$) 만 주고 컴퓨터에게 수많은 데이터 속에 숨겨져 있는 패턴을 찾도록 한다. 연관/순차규칙 분석, 군집화, 차원 축소, 네트워크 분석 등이 비지도 학습에 속한다. 그런데 비지도 학습은 Y값이 없어, 학습 종료 후에 도출결과를 객관적으로 평가하기가 다소 애매하며 분석가의 주관이 들어갈 수 있다.

: Classifier, Clustering ...

* 지도학습 / 비지도학습 비교 *

분류	지도학습	비지도학습
주관	사람이 개입	컴퓨터 자체
기법	확률과 통계 기반 추론 통계	패턴분석 기반, 데이터 마이닝
유형	회귀분석, 신경망 (y변수 O)	군집분석, 연관분석 (y변수 X)

기계학습 방법	가능 분야	분석방법의 종류 및 알고리즘
지도학습 (Supervised Learning)	예측, 추정 (Prediction, Estimation)	✧ Linear Regression ✧ Regression Tree, Model Tree ✧ SVM(Support Vector Machine) ✧ Neural Network, Deep Learning ✧ ARIMA, Exponential Smoothing
	분류 (Classification)	✧ Decision Tree ✧ Logistic Regression, Discriminant Analysis ✧ k-NN(k-Nearest Neighbor), CBR(Case-Based Reasoning) ✧ Naïve Bayes Classification ✧ SVM, Neural Network ✧ Ensemble (Bagging, Boosting, Random Forest)
비지도학습 (Unsupervised Learning)	패턴/구조 발견 (Pattern/Rule)	✧ Association Rule Analysis, Sequence Analysis ✧ Network Analysis, Link Analysis, Graph theory ✧ Structural Equation Modeling, Path Analysis
	그룹화 (Grouping)	✧ k-Means Clustering, Hierarchical Clustering, Density-based Clustering, Fuzzy Clustering ✧ SOM(Self-Organizing Map)
	차원 축소 (Dimension Reduction)	✧ PCA(Principal Component Analysis), Factor Analysis, SVD(Singular Value Decomposition)
	영상, 이미지, 문자 (Video, Image, Text, Signal processing)	✧ Wavelet/Fast Fourier Transformation, DTW(Dynamic Time Warping), SAX(Symbolic Aggregate Approximation), Line/Circular Hough Transformation ✧ Text mining, Sentiment Analysis

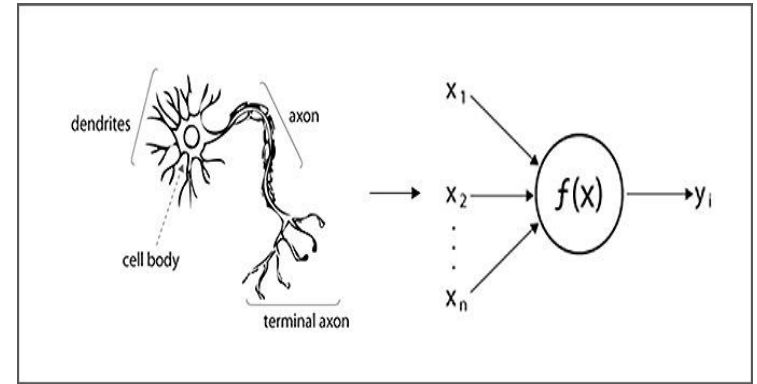
Neural Network

신경망(neural network) 기법은 반복적인 학습 과정을 거쳐 데이터에 내재되어 있는 패턴을 찾아내고 이를 일반화함으로써 대용량 데이터로부터 의사결정에 필요한 유용한 정보를 찾아내는 블랙박스 기법이다. 여러 분야의 다양한 문제에 적용될 수 있고, 독립변수와 종속변수의 관계를 살피기가 어려운 복잡한 데이터에 대해서도 좋은 결과를 주는 것으로 알려져 있다. 하지만, 분류나 예측 결과만을 제공할 뿐 결과가 어떻게 나왔는가에 대한 이유를 설명하지 못한다. 따라서 신경망은 데이터의 여러 변수들간의 관계를 밝혀내는 데는 적합하지 않으며, 예측이 필요한 경우에 이용될 수 있다.

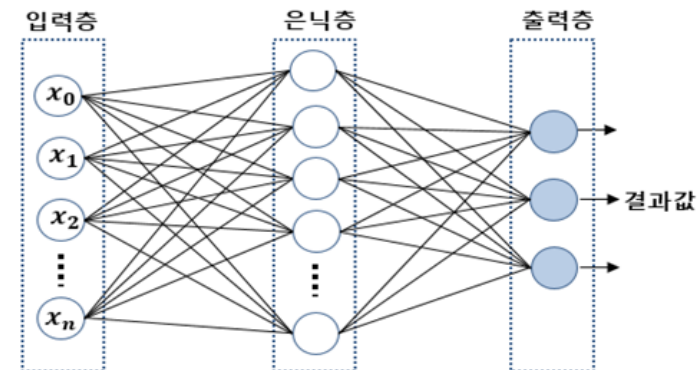
신경망 분석의 장단점

장점	적용 가능한 문제의 영역이 넓음	o 입력, 출력마디에 이산형, 연속형 변수 모두 사용가능하며 기법을 적용할 수 있는 문제의 영역이 Decision Tree나 통계에 비해 넓음
	제품이 많음	o 상용화된 데이터마이닝 제품이 많으며, 제품 선택의 폭이 넓음
단점	과정에 대한 설명부족	o 분류나 예측결과만 제공할 뿐 결과에 대한 근거를 설명하지 못함
	모형 구축의 어려움	o 복잡한 학습과정을 거치기 때문에 모형 구축시 많은 시간이 소요. 따라서 입력 변수의 수가 너무 많으면 통계나 Decision Tree를 이용, 변수 선별 후 구축하는 대안을 고려 할 수 있음
	전문가 필요	o 다양한 Parameter값을 설정하는 작업이 전문성을 필요로 하기 때문에 비전문가들이 사용하기 어려움

생물학적 신경망을 모방하여 인공신경망을 모델링한 내용을 보면 처리 단위 측면에서는 생물적인 neurons가 nodes로, Connections은 Synapse가 가중치로 모델링되었다.



다층 신경망 (MLP)



Keras

Keras는 머신러닝 라이브러리 Theano와 TensorFlow를 Wrapping한 Deep Learning 라이브러리다.

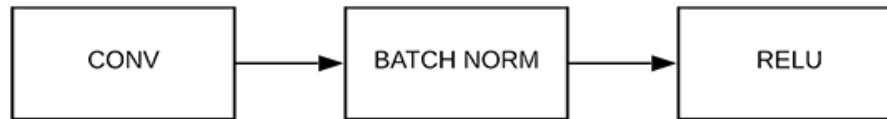
- 사용자 친화적. 일관성 있고 간단한 API를 제공
- 모듈식, 구성 가능. 머신러닝 모델을 간단하게 작성할 수 있다.
- 손쉬운 확장
- 단점 : 불충분한 에러 메시지. (많이 보완되고 있음.)
프레임워크 구조가 복잡하다.
규모가 큰 모델은 수행시간이 많이 걸린다.
- TensorFlow 도 keras 라이브러리를 사용하고 있다. tf.keras 형태로 사용 가능.

<https://keras.io>

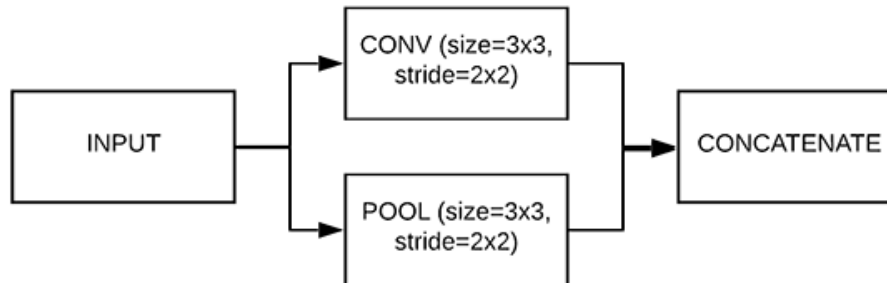
<https://2sang.github.io/journal/tf-devsummit-2.html>

3 ways to create a Keras model with TensorFlow 2.0 (Sequential, Functional, and Model Subclassing)

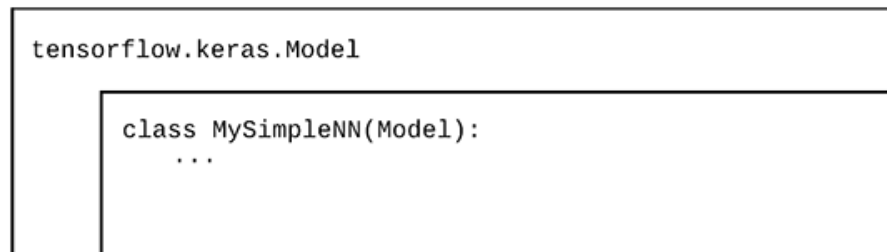
1. Sequential API



2. Functional API



3. Model Subclassing



케라스(Keras) 기본 개념

- 가장 핵심적인 데이터 구조는 "모델"이다.
- 시퀀스 모델로 원하는 Layer를 쉽게 순차적으로 쌓을 수 있다.
- 신경망을 구성하기 위한 각 구성요소를 클래스로 제공하기 때문에 쉽게 신경망을 구현 가능

케라스 모델링 순서

1. 데이터 셋 생성

원본 데이터를 불러오거나 데이터를 생성한다. 데이터로부터 TrainSet, ValidationSet, TestSet을 생성. 이 때 딥러닝 모델의 학습 및 평가를 할 수 있도록 포맷 변환을 한다.

2. 모델 구성

시퀀스 모델을 생성한 뒤 필요한 Layer를 추가하며 구성.
좀 더 복잡한 모델이 필요할 때는 케라스 함수 API를 이용한다

3. 모델 학습과정 설정

학습하기 전, 학습에 대한 설정을 수행한다.
손실 함수 및 최적화 방법을 정의. **compile()** 함수를 사용한다.

4. 모델 학습시키기

TrainSet을 이용하여 구성한 모델로 학습시킨다. **fit()** 함수를 사용한다.

5. 학습과정 살펴보기

모델 학습 시 TrainSet, ValidationSet 의 손실 및 정확도를 측정한다.
반복횟수에 따른 손실 및 정확도 추이를 보면서 학습상황 판단.

6. 모델 평가

준비된 TestSet 으로 학습한 모델을 평가한다. **evaluate()** 함수를 사용.

7. 모델 사용하기

임의의 입력으로 모델의 출력을 얻는다. **predict()** 함수를 사용한다.

1. DataSet 생성

2. 모델 구성

```
model = Sequential()
```

```
model.add(Dense(1, input_dim=3, init='uniform', activation='relu'))
```

위 코드에서 Dense()는 한번 사용되었지만 더 많은 레이어를 추가할 수 있다.
첫 번째 인자의 값은 1인데 이는 총 1개의 출력 뉴런을 의미한다.
두 번째 인자인 input_dim은 입력층의 뉴런 수를 의미한다.
3개의 입력층 뉴런과 1개의 출력층 뉴런을 만들었다.

Dense()의 주요 인자.

첫 번째 인자 = 출력 뉴런의 수.

input_dim = 입력 뉴런의 수. (입력의 차원)

init = 가중치 초기화 방법.

- uniform : 균일 분포, normal : 가우시안 분포

activation = 활성화 함수.

- linear : 디폴트 값으로 별도 활성화 함수 없이 입력 뉴런과 가중치의 계산 결과 그대로 출력.
- sigmoid : 이진 분류 문제에서 출력층에 주로 사용되는 활성화 함수.
- softmax : 셋 이상을 분류하는 다중 클래스 분류 문제에서 출력층에 주로 사용되는 활성화 함수.
- relu : 은닉층에 주로 사용되는 활성화 함수.

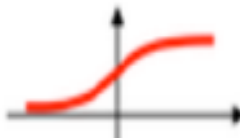
* keras에는 다양한 layer들이 담겨있다.

- Convolutional layers : Conv1D, Conv2D, Conv3D, Conv2DTranspose, etc.
- Pooling layers : MaxPooling1D, MaxPooling2D, MaxPooling3D, AveragePooling1D, etc.
- RNN layers : GRU, LSTM, ConvLSTM2D, etc.
- BatchNormalization, Dropout, Embedding, etc.

▪ 활성화 함수 (Activation Function)

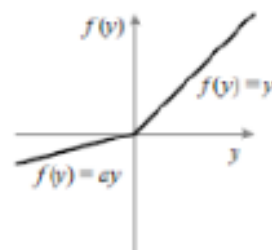
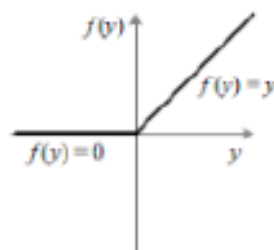
- Synapse는 전달된 전기 신호가 최소한의 자극 값을 초과하면 활성화되어 다음 뉴런으로 전기신호를 전달한다.
- 활성화 함수는 이것을 모방하여 값이 작을 때는 출력 값을 작은 값으로 막고, 일정한 값을 초과하면 출력 값이 급격히 커지는 함수를 이용한다.
- 신경망에서 Node는 전달받은 데이터를 가중치를 고려해 합산하고, 그 값을 활성화 함수를 적용해 다음 층에 전달한다.

▪ 은닉층에서 자주 이용되는 활성화 함수

tanh 함수 (Hyperbolic Tangent Function, 쌍곡 탄젠트 함수)	 $y = \tanh(x)$
Sigmoid 함수	 $y = \frac{1}{1 + e^{-x}}$
ReLU 함수 (Rectified Linear Unit Function)	 $y = 0 \text{ (if, } x < 0)$ $y = x \text{ (if, } x \geq 0)$

▪ Parametric ReLU 함수

- ReLU함수는 0보다 작은 값에서 경사가 0이다.
- 그래서 이 부분에 작은 경사 값을 준 **Parametric ReLU**함수도 자주 이용된다.



- 예전에는 활성화 함수로 tanh함수나 Sigmoid함수를 자주 사용했으나 최근에는 ReLU 함수를 사용
→ ReLU 함수는 상대적으로 학습 속도가 빠르고 학습이 잘 됨.

3. 모델 학습과정 설정

학습하기 전, 학습에 대한 설정을 수행한다.

손실 함수 및 최적화 방법을 정의. 케라스에서는 compile() 함수를 사용한다.

예) **model.compile(optimizer='rmsprop', loss='binary_crossentropy', metrics=['acc'])**

위 코드는 임베딩층, 은닉층, 출력층을 추가하여 모델을 설계한 후에, 마지막으로 컴파일 하는 과정을 진행.

속성 :

optimizer : 훈련 과정을 설정하는 옵티마이저를 설정. 'adam'이나 'sgd'와 같이 문자열로 지정 가능.
SGD(), RMSprop(), Adam() etc.

loss : 훈련 과정에서 사용할 손실 함수(loss function)를 설정.

metrics : 훈련을 모니터링 하기 위한 지표를 선택.

손실 함수 : 성공 지표에 대한 손실 함수로 바꿀 수 없는 경우도 있음(ROC, AUC)

문제 유형	마지막 층 activation	loss func
binary classification	sigmoid	binary_crossentropy
단일 label multi classification	softmax	categorical_crossentropy
다중 label multi classification	sigmoid	binary_crossentropy
임의 값에 대한 regression	없음	mse(mean squared error)
0과 1 사이 값에 대한 regression	sigmoid	mse or binary_crossentropy

4. 모델 학습시키기

train set을 이용하여 구성한 모델로 학습시킨다. Keras에서는 fit() 함수를 사용한다.

Ex) **model.fit(X_train, y_train, epochs=10, batch_size=32)**

속성 : 첫번째 인자 = 훈련 데이터에 해당됨.

두번째 인자 = 지도 학습 관점에서 레이블 데이터에 해당됨.

epochs = epochs 1은 전체 데이터를 한 차례 훑고 지나갔음을 의미함. 정수값 기재 필요.
총 훈련 횟수를 정의.

batch_size = 기본값은 32. 미니 배치 경사 하강법을 사용하고 싶지 않을 경우에는
batch_size=None을 통해 선택 가능.

5. 학습과정 살펴보기

모델 학습 시 TrainSet, ValidationSet 손실 및 정확도를 측정한다.

반복횟수에 따른 손실 및 정확도 추이를 보면서 학습상황 판단.

6. 모델 평가

준비된 TestSet으로 학습한 모델을 평가한다. Keras에서는 evaluate() 함수를 사용

Ex) **model.evaluate(X_test, y_test, batch_size=32)**

속성 : 첫번째 인자 = 테스트 데이터에 해당됨.

두번째 인자 = 지도 학습 관점에서 레이블 테스트 데이터에 해당됨.

batch_size = 배치 크기.

7. 모델 사용하기

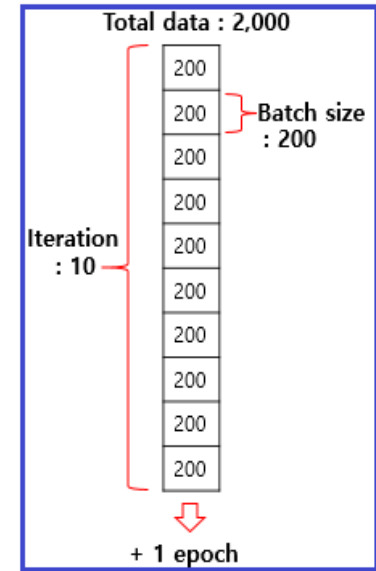
임의의 입력으로 모델의 출력을 얻는다. Keras에서는 predict() 함수를 사용한다.

Ex) **model.predict(X_input, batch_size=32)**

속성 : 첫번째 인자 = 예측하고자 하는 데이터. batch_size = 배치 크기.

Epochs and Batch size and Iteration

컴퓨터는 실제값과 예측값의 오차로부터 옵티마이저를 통해서 가중치를 업데이트한다. 머신 러닝에서는 이 과정을 **학습**이라고 한다. 이를 현실의 학습에 비유하면 사람은 문제지의 문제를 풀고, 정답지를 보며 채점을 하면서 부족했던 점을 깨달으며 지식이 업데이트되는 과정이다. 하지만 사람마다 동일한 문제지와 정답지를 주더라도 공부 방법은 천차만별이다. 어떤 사람은 문제지 하나를 다 풀고 나서 정답을 채점하는데 어떤 사람은 문제지의 문제를 10개 단위로 끊어서 공부한다. 문제 10개를 풀고 채점하고 다시 다음 문제 10개를 풀고 채점하고 반복하는 방식으로 학습한다는 것이다. 게으른 사람은 문제지를 3번 공부하는데, 성실한 사람은 문제지의 문제를 외울 만큼 문제지를 여러 차례 공부한다. 컴퓨터도 똑같다. 같은 문제지와 정답지를 주더라도 공부 방법을 다르게 설정할 수 있다. 그림은 에포크와 배치 크기와 이터레이션의 차이를 보여준다.



- Epoch

인공 신경망에서 전체 데이터에 대해서 순전파와 역전파가 끝난 상태를 말한다. 만약 에포크가 50이라고 하면, 전체 데이터 단위로는 총 50번 학습한다. 이 에포크 횟수가 지나치거나 너무 적으면 앞서 배운 과적합과 과소적합이 발생할 수 있다.

- Batch size

배치 크기는 몇 개의 데이터 단위로 매개변수를 업데이트 하는지를 말한다. 현실에 비유하면 문제지에서 몇 개씩 문제를 풀고 나서 정답지를 확인하느냐의 문제다. 사람은 문제를 풀고 정답을 보는 순간 부족했던 점을 깨달으며 지식이 업데이트 된다고 하였다. 컴퓨터는 실제값과 예측값으로부터 오차를 계산하고 옵티마이저가 매개변수를 업데이트 한다. 여기서 중요한 포인트는 업데이트가 시작되는 시점이 정답지/실제값을 확인하는 시점이라는 것이다. 사람이 2,000 문제가 수록되어있는 문제지의 문제를 200개 단위로 풀고 채점한다고 하면 이때 배치 크기는 200 이다. 기계는 배치 크기가 200이면 200개의 샘플 단위로 가중치를 업데이트 한다.

주의할 점은 배치 크기와 배치의 수는 다른 개념이라는 점이다. 전체 데이터가 2,000일때 배치 크기를 200으로 준다면 배치의 수는 10 이다. 이는 에포크에서 배치 크기를 나눠준 값($2,000/200 = 10$)이기도 하다. 이때 배치의 수를 이터레이션이라고 한다.

- Iteration

한 번의 에포크를 끝내기 위해서 필요한 배치의 수를 말한다. 또는 한 번의 에포크 내에서 이루어지는 매개변수의 업데이트 횟수이기도 하다. 전체 데이터가 2,000일 때 배치 크기를 200으로 한다면 이터레이션의 수는 총 10개다. 이는 한 번의 에포크 당 매개변수 업데이트가 10번 이루어진다는 것을 의미한다. SGD를 이 개념을 가지고 다시 설명하면, SGD는 배치 크기가 1이므로 모든 이터레이션 마다 하나의 데이터를 선택하여 경사 하강법을 수행한다.

**** Regularization과 딥러닝의 일반적인 흐름 ****

- 최적화(optimization) : 가능한 훈련 데이터에서 최고의 성능을 얻으려고 모델을 조정하는 과정
- 일반화(generalization) : 훈련된 모델이 이전에 본 적 없는 데이터에서 얼마나 잘 수행되는지를 의미
- 과소적합(underfitting) : 훈련 데이터의 손실이 낮아질수록 테스트 데이터의 손실도 낮아진다. 즉, 모델 성능이 계속 발전될 여지가 있음.
- 과대적합(overfitting) : 모델이 훈련 데이터에 특화되어 일반화 성능이 높아지지 않고 오히려 감소

Regularization : 과대적합을 피하는 처리 과정

가장 좋은 방법은 더 많은 훈련 데이터를 모으는 것
차선택은 모델이 수용할 수 있는 정보의 양을 조절하거나 저장할 수 있는 정보에 제약을 가하는 것
모델에 있는 학습 파라미터의 수를 줄이는 것 (학습되는 가중치) - 파라미터의 수는 층의 수와 각 층의 유닛 수에 의해 결정된다.
적절한 layer 수와 capacity 크기를 결정해야 됨. 작게 시작해서 검증 손실에 따라 크게 하면서 모델 구축

weight regularization(가중치 규제) : 네트워크의 복잡도에 제한을 두어 가중치가 작은 값을 가지도록 강제하는 것
(이렇게 하면 가중치 값의 분포가 더 균일하게 됨)

L1 regularization : 가중치의 절댓값에 비례하는 비용이 추가됨(가중치의 L1 norm)

L2 regularization(=weight decay) : 가중치의 제곱에 비례하는 비용이 추가됨(가중치의 L2 norm)

`regularizers.l2(0.001)` : 가중치 행렬의 모든 원소를 제공하고 0.001을 곱하여
네트워크의 전체 손실에 더해진다는 의미.
이 규제(패널티)는 훈련할 때만 추가됨.

```
model.add(layers.Dense(
    16,
    kernel_regularizer=
        regularizers.l2(0.001),
    activation='relu',
    input_shape=(10000,)
))
```

Drop out : 입력 벡터 중 일부를 무작위로 0으로 바꿈,
Dropout 비율 : 0이 될 특성의 비율, 비율이 높을수록 0이 많이 됨, 보통 0.2~0.5 사이

가중치 규제하기 : 오컴의 면도날(Occam's Razor) 이론은 어떤 것을 설명하는 두 가지 방법이 있다면 더 정확한 설명은 최소한의 가정이 필요한 가장 "간단한" 설명일 것이다. 라는 개념이다. 이는 신경망으로 학습되는 모델에도 적용된다. 훈련 데이터와 네트워크 구조가 주어졌을 때 이 데이터를 설명할 수 있는 가중치의 조합(즉, 가능한 모델)은 많다. 간단한 모델은 복잡한 것보다 과대적합 되는 경향이 작을 것이다. "간단한 모델"은 모델 파라미터의 분포를 봤을 때 entropy가 작은 모델이다. 또는 적은 파라미터를 가진 모델이다. 따라서 과대적합을 완화시키는 일반적인 방법은 가중치가 작은 값을 가지도록 네트워크의 복잡도에 제약을 가하는 것이다. 이는 가중치 값의 분포를 좀 더 균일하게 만들어 준다. 이를 "가중치 규제"(weight regularization)라고 부른다. 네트워크의 손실 함수에 큰 가중치에 해당하는 비용을 추가한다.

- 이 비용은 두 가지 형태가 있다.
- 1) L1 규제 : 가중치의 절댓값에 비례하는 비용이 추가된다(즉, 가중치의 "L1 노름(norm)"을 추가).
 - 2) L2 규제 : 가중치의 제곱에 비례하는 비용이 추가된다(즉, 가중치의 "L2 노름"의 제곱을 추가).
- 참고로 신경망에서는 L2 규제를 가중치 감쇠(weight decay)라고도 부른다.

Model의 저장과 복원

가중치 : `tf.keras.Model.save_weights`를 사용하여 모델의 가중치를 저장하고 복원한다.

가중치를 텐서플로의 체크포인트 파일로 저장.

```
model.save_weights('./weights/my_model')
```

모델의 상태를 복원한다. 모델의 구조가 동일해야 한다.

```
model.load_weights('./weights/my_model')
```

기본적으로 모델의 가중치는 텐서플로 체크포인트 파일 포맷으로 저장된다.

이와는 달리 케라스는 기본적으로 .hdf5 형식을 사용한다

회귀분석(Regression Analysis)

변수 간의 인과관계를 밝히기란 매우 어려운 문제다. 수학적 방법 이외에 다양한 외적 조건도 따져봐야 한다.

회귀분석은 이런 과정 중에 하나에 불과하다.

- 특정변수(독립변수)가 다른 변수(종속변수)에 어떤 영향을 미치는가를 분석. 즉, 인과관계를 분석.
- 독립, 종속변수는 등간 또는 비율척도(연속형 데이터)로 구성되어야 한다. 한 변수의 변화에 따른 다른 변수의 값을 파악.
- 독립변수가 종속변수에 영향을 미치는 변수를 규명하고, 이 둘 변수들에 의해서 회귀방정식을 도출하여 회귀선을 추정한다. $y = wx + b$
- 회귀분석은 시간에 따라 변화하는 데이터나 어떤 영향, 가설적 실험, 인과 관계의 모델링 등의 통계적 예측에 이용 가능.
- 회귀분석의 기본 가정 충족 조건 : 선형성, 잔차 정규성, 잔차 등분산성, 잔차 독립성, 다중 공선성 등

~ 상관계수 분석 : 변수 간의 **관련성** 분석

~ 회귀분석 : 변수 간의 **인과관계** 분석, 사용범위가 넓은 분석방법이다

1) **단순회귀분석** : 독립변수와 종속변수가 각각 1개인 경우에 독립변수가 종속변수에 어떠한 영향을 미치는지 인과관계를 분석.

- 연구모델 : 제품적절성(독립변수) -> 제품 만족도(종속변수)

2) **다중회귀분석**: 여러 개의 독립변수로 1개의 종속변수에 미치는 영향 분석

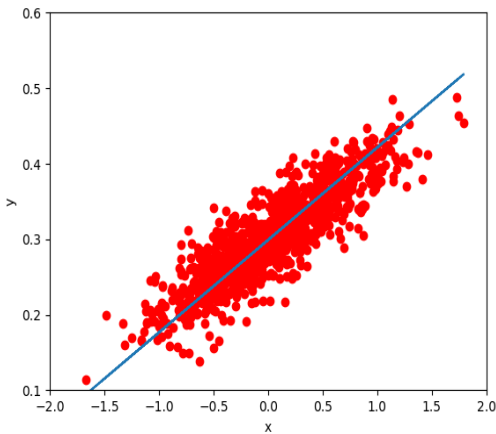
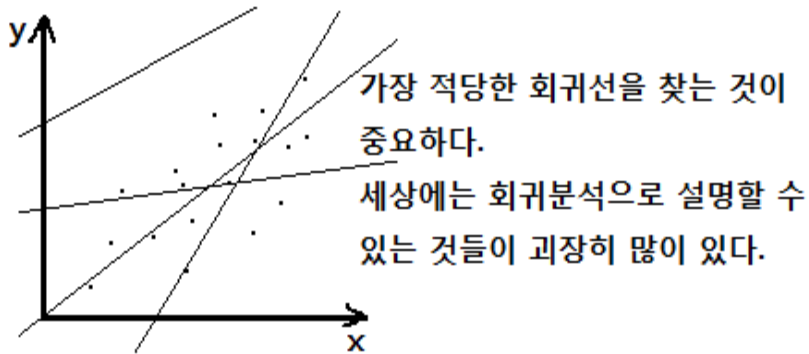
- 연구모델 : 제품 적절성, 제품 친밀도 -> 제품 만족도

회귀분석(Regression Analysis)

변수들 사이의 관계를 분석하는데 사용하는 통계학적 방법

Hypothesis : $H(x) = Wx + b$ or $y = W * x + b$ 라고도 표현함.

Weight와 bias 의 값에 따라 선의 모양이 달라진다. W와 b의 값을 최소화하도록 학습모델을 작성한다. 학습을 통해 W와 b의 최소값을 얻게 된다.



$$\text{Cost(loss) function} = \frac{1}{N} \sum_{i=1}^n (W^{(i)} - y^{(i)})^2$$

Minimize 되도록 학습

학습데이터 수

예측값

실제값

Cost function을 이용하면 오차 측정이 가능하다.

비용함수란 신경망 학습에서 학습 데이터에 대한 오차를 측정하는 척도가 된다.

Cost function : 생성한 hypothesis 함수의 정확도를 측정하기 위한 함수.

ex1) 텐서플로우로 단순선형회귀 - 경사하강법 함수 사용 1.x

```
import tensorflow.compat.v1 as tf # tensorflow 1.x 소스 실행시
tf.disable_v2_behavior()          # tensorflow 1.x 소스 실행시
```

```
x_data = [1.,2.,3.]
y_data = [1.,2.,3.]
```

```
x = tf.placeholder(tf.float32)
y = tf.placeholder(tf.float32)
```

```
w = tf.Variable(tf.random_normal([1]))
b = tf.Variable(tf.random_normal([1]))
```

```
hypothesis = x * w + b
cost = tf.reduce_mean(tf.square(hypothesis - y))
```

```
print('\n경사하강법 메소드 사용-----')
optimizer = tf.train.GradientDescentOptimizer(learning_rate=0.01)
train = optimizer.minimize(cost)
```

```
sess = tf.Session() # Launch the graph in a session.
sess.run(tf.global_variables_initializer())
w_val = []
cost_val = []
```

```
for i in range(501):
    _, curr_cost, curr_w, curr_b = sess.run([train, cost, w, b],
                                             {x:x_data, y:y_data})
    w_val.append(curr_w)
    cost_val.append(curr_cost)
    if i % 10 == 0:
        print(str(i) + ' cost:' + str(curr_cost) + ' weight:' +
              str(curr_w) + ' b:' + str(curr_b))
```

ex2) 텐서플로우로 회귀분석 기본 - Keras 사용 2.x

```
from keras.models import Sequential
from keras.layers import Dense
from keras import optimizers
```

```
x_data = [1.,2.,3.]
y_data = [1.,2.,3.]
model=Sequential() # 계층구조를 이루는 모델을 정의
model.add(Dense(1, input_dim=1, activation='linear'))
sgd=optimizers.SGD(lr=0.01) # 학습률(learning rate, lr)은 0.01
model.compile(optimizer=sgd, loss='mse', metrics=['mse'])
```

```
# 주어진 X와 y데이터에 대해서 오차를 최소화하는 작업을 100번 시도.
model.fit(x_data, y_data, batch_size=1, epochs=100,
          shuffle=False, verbose=1)
```

```
print('예상 수 : ', model.predict([5])) # [[4.801656]]
print('예상 수 : ', model.predict([2.5])) # [[2.490468]]
print('예상 수 : ', model.predict([1.5, 3.3])) # [[1.565993] [3.230048]]
```

Callback Object

Callback은 훈련하는 동안 모델의 동작을 변경하고 확장하기 위해 전달하는 객체다. 자신만의 콜백을 작성하거나 다음과 같은 내장 `tf.keras.callbacks`를 사용할 수 있다.

- `tf.keras.callbacks.ModelCheckpoint` : 일정 간격으로 모델의 체크포인트를 저장.
(가장 좋은 모델의 가중치, loss, acc 등)
- `tf.keras.callbacks.LearningRateScheduler` : 학습률을 동적으로 변경.
- `tf.keras.callbacks.EarlyStopping` : 검증 성능이 향상되지 않으면 훈련을 중지.
- `tf.keras.callbacks.TensorBoard` : 텐서보드를 사용하여 모델을 모니터링.

`fit` 메서드에 매개변수로 전달하여 사용한다.

```
callbacks = [  
    # 'val_loss'가 2번의 에포크에 걸쳐 향상되지 않으면 훈련을 멈춘다.  
    tf.keras.callbacks.EarlyStopping(patience=2, monitor='val_loss'),  
  
    # './logs' 디렉토리에 텐서보드 로그를 기록.  
    tf.keras.callbacks.TensorBoard(log_dir='./logs')  
]
```

```
model.fit(data, labels, batch_size=32, epochs=5, callbacks=callbacks,  
          validation_data=(val_data, val_labels))
```

TensorBoard로 그래프 진행을 모니터링

TensorBoard 내의 그래프 결과 확인 (localhost:6006)

TensorBoard

SCALARSIMAGESAUDIO**GRAPHS**

Fit to screen

Download PNG

Run

C\test4

(6)

Session runs (0)

Upload

Choose File

Trace inputs

Color

Structure

Main Graph

Auxiliary Nodes

TensorBoard symbol

Symbol	Meaning
	High-level node representing a name scope. Double-click to expand a high-level node.
	Sequence of numbered nodes that are not connected to each other.
	Sequence of numbered nodes that are connected to each other.
	An individual operation node.
	A constant.
	A summary node.
	Edge showing the data flow between operations.
	Edge showing the control dependency between operations.
	A reference edge showing that the outgoing operation node can mutate the incoming tensor.

로지스틱 회귀분석 (Logistic Linear Regression)

종속변수와 독립변수 간의 관계를 나타내어 예측모델을 생성한다는 점에서 선형회귀분석 방법과 유사하다. 하지만 독립변수(x)에 의해서 종속변수(y)의 범주로 분류한다는 측면은 분류분석 방법으로 간주된다.

특징

- 1) 분석 목적 : 종속변수(y 범주형)와 독립변수(x 연속형) 간의 관계를 통해서 예측모델을 생성하는데 있다.
- 2) 회귀분석과 차이점 : 종속변수는 반드시 범주형이어야 한다.
(이항형 : Yes/No 또는 다항형 : iris의 Spices 칼럼)
- 3) 정규성 : 정규분포 대신에 이항분포를 따른다.
- 4) 로짓변환 : 종속변수의 출력범위를 0과 1로 조정하는 과정을 의미한다.
예) 혈액형 A인 경우 => [1,0,0,0]
- 5) 활용분야 : 의료, 통신, 날씨 등 다양한 분야에서 활용

로지스틱 회귀분석은 선형회귀분석과 달리 종속변수(y 결과변수)가 범주형 데이터인 경우에 사용되는 기법이다. 선형회귀분석 모델에서는 독립변수(x 설명변수)를 입력하면 수치형 결과를 얻게 된다.

그러므로 신장, 시험성적, 연간소득 따위를 예측하고 싶은 경우에는 선형분석을 사용하면 된다. 이와 달리 예측하고자 하는 것이 수치화 하기 힘든 변수, 예를 들어 어떤 고객이 부도를 낼 것인지의 여부, 타이타닉호에서 살아남을 것인지 여부, 어떤 사람의 최종학력 알기 등의 경우에는 로지스틱 회귀분석을 사용하면 된다. 회귀분석의 일종이므로 지도학습에 해당된다.

로지스틱 회귀분석

Logistic regression은 Linear regression과 다르게 결과를 0과 1로 나타낸다. 이 때 사용하는 함수는 sigmoid 함수다.

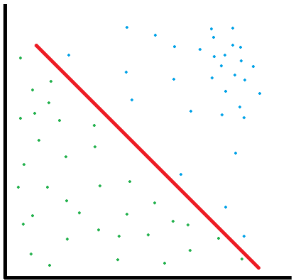
가설의 형태가 sigmoid 함수로 나타나게 되는데 들어가는 x 의 값에 weight이 곱해지는 형태가 된다.

cost function의 경우 log를 이용하여 Linear regression과는 다르게 정의를 하게 된다.

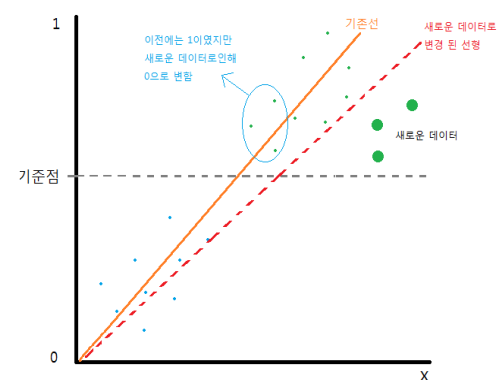
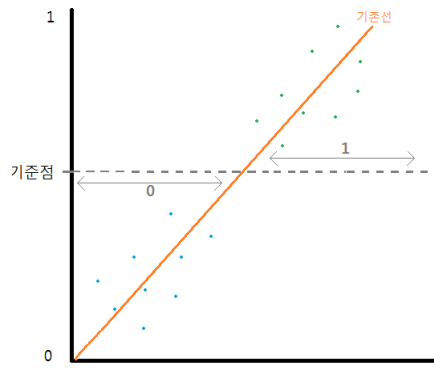
하지만 이 cost function 역시 Gradient descent algorithm을 사용하여 최소화 시킨다.

Classification(분류)은 값이 두 개만 존재하는 이항분류와 다항분류로 구분할 수 있다.

로지스틱 회귀분석은 이항분류(binary classification) 모델이다.



로지스틱 분류는 좌측 그림처럼 데이터를 구분 지을 수 있는 이상적인 선을 긋는 것이라고 할 수 있다. 이항분류를 Linear Regression(선형 회귀)를 이용하여 접근할 경우문제가 생길 수 있다. 예를 들어 다른 학습 데이터가 추가되었다고 가정을 할 경우 기준점이 변경될 수 있다.



그림과 같이 어떤 데이터들이 있었다고 가정을 한다.

기준점은 0.5라고 하고 그 기준점에 따라 선형을 좌우로 구분하여 좌측의 데이터는 0, 우측의 데이터는 1이 되는 결과를 얻을 수 있다. 문제는 위의 상태에서 새로운 데이터가 추가될 경우에 발생한다.

One-hot Encoding 이란?

다음과 같이 3가지 꽃의 종류가 있다고 가정하자.

0: setosa, 1: versicolor, 2: virginica

3가지의 꽃이 데이터 셋에 각각 해당 타입의 숫자의 배열로 되어 있다.

예) [0, 1, 2, 1, 2, 0, 0, 0, 1, 2, ...]

머신러닝을 할 때, 위와 같이 0, 1, 2의 데이터가 들어간다면, setosa / versicolor / virginica는 서로 관계 없는 꽃의 종류일지라도 숫자로 표현 되어있기 때문에 학습을 할 때 영향을 미칠 수 있다.

예를 들어 설명하자면 $1 + 1 = 2$ 라는 수식에 의해

versicolor + versicolor = virginica 이라는 결과를 초래할 수 있다는 점이다.

이를 방지하기 위해 꽃의 종류가 3 가지라면, 별도의 3개의 column을 만들고

3개의 column 중 해당 타입의 column 에만 1, 다른 column은 0을 대입해주는 pre-processing 을 거치면 된다. 이를 one-hot encoding 기법이라 한다.

바뀐 예 :
[1, 0, 0]
[0, 1, 0]
[0, 0, 1]
[0, 1, 0]
[0, 0, 1]
...

Softmax 분류모델

SVM과 더불어 많이 쓰이는 분류모델이다. Logistic regression classifier의 multiple class에 대한 일반화 개념으로 간단히 logistic을 일반화한 것이라고 보면 된다. logistic은 계산을 해서 1, 0만 나누었다면 softmax는 같은 계산을 해서 각 class별로 확률을 매긴다. 가장 높은 확률의 편을 들어주는 것이다.

텐서플로우가 아니라면 softmax 부분을 어떻게 구현해야 할까? 일일이 class별로 weight와 bias를 계산해서 sigmoid 씌우고 어쩌구 저쩌구... 끔찍하다. tensorflow에서는 메서드로 이를 제공한다.

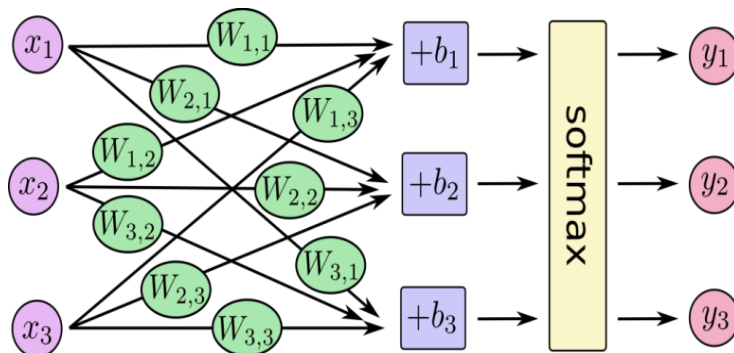
아래의 그림을 보자!

각 weight를 곱하고 bias를 더한 것을 0~1까지의 확률로 자동적으로 표현해 준다.

한 마디로 각 영역별로 시그모이드 함수를 구현하는 부분을 우리가 구현할 필요가 없게 된다.

hypothesis, 즉 기대 값은 x 와 W 의 행렬곱을 softmax 함수로 처리 한 것이다.

cost함수는 오류를 극대화 하기 위해서 + 선형성을 위해서 저렇게 한 것이라고 생각하면 된다.



출처: 텐서플로우코리아

※ **Softmax function**은 logistic regression이 2가지 레이블 분류하는 것과 달리 여러 개의 label에 대해서도 분류(multiclass classification)를 가능하게 한다. 2개의 레이블에 대한 분류에 대해 Softmax function과 logistic function은 같다(동치). softmax는 점수로 나온 결과를 전체 합계가 1이 되는 0 ~ 1 사이의 값으로 변경해 준다. 전체를 더하면 1이 되기 때문에 확률(probabilities)이라고 부르면 의미가 더욱 분명해진다.

CNN(Convolutional Neural Networks)

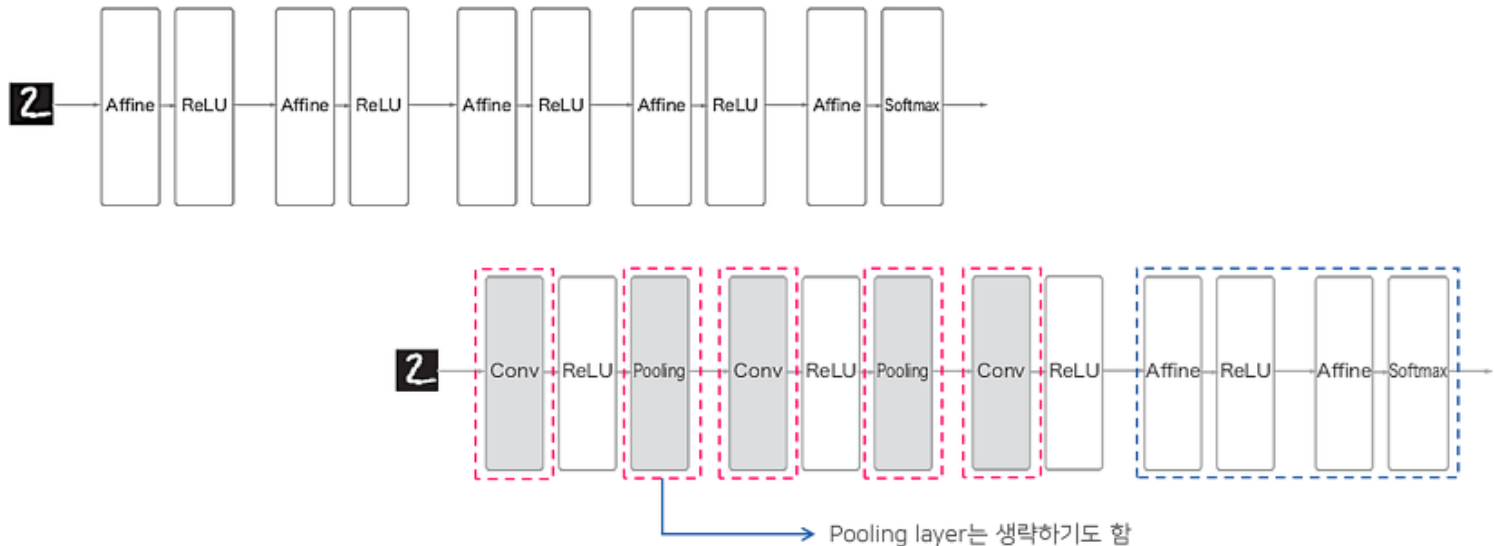
CNN은 1989년 LeCun이 발표한 논문 "Backpropagation applied to handwritten zip code recognition"에서 처음 소개됨

2003년 Behnke의 논문 "Hierarchical Neural Networks for Image Interpretation"을 통해 일반화 됨

Simard의 논문 "Best Practices for Convolutional Neural Networks Applied to Visual Document Analysis"에서 단순화

CNN의 구조

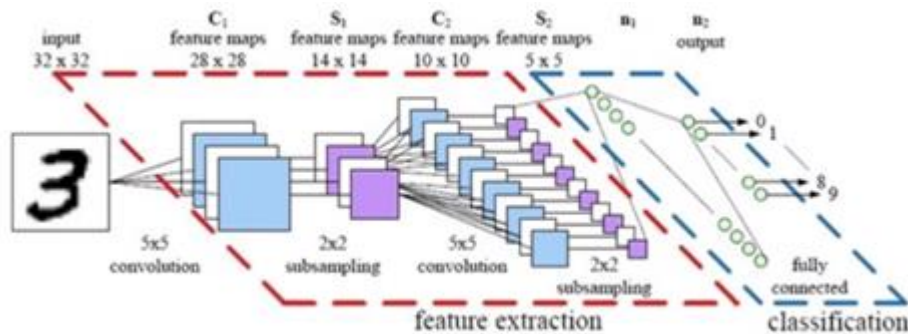
CNN의 구조는 완전연결계층(Fully-Connected Layer)과는 다른 점이 있다. Fully-Connected Layer에서는 Affine계층으로 구현했지만, CNN에서는 아래의 그림과 같이 합성곱 계층(Convolutinal Layer) 과 풀링 계층(Pooling Layer)이 추가된다.



CNN

합성곱 계층(Convolutional Layer, Conv Layer)

MNIST 데이터 같은 이미지 데이터는 일반적으로 **채널, 세로, 가로** 이렇게 3차원으로 구성된 데이터이다. Affine 계층에서는 이 3차원 데이터를 1차원 데이터($784=28 \times 28$)로 바꿔 입력했지만 합성곱에서는 3차원 데이터(1, 28, 28)를 입력하고 3차원의 데이터로 출력하므로 형상을 유지할 수 있다. CNN에서는 이러한 입출력 데이터를 특징 맵(Feature Map)이라 한다.



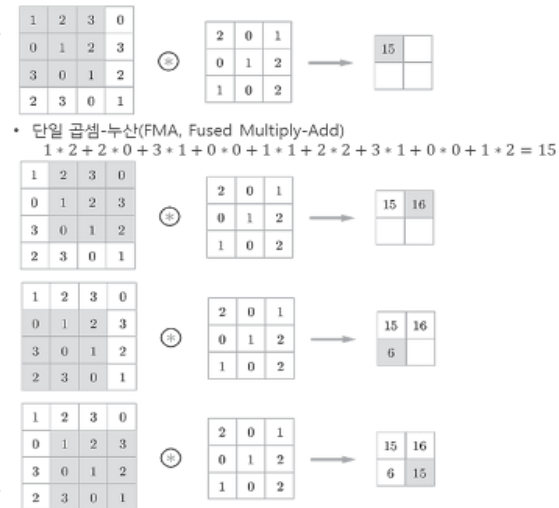
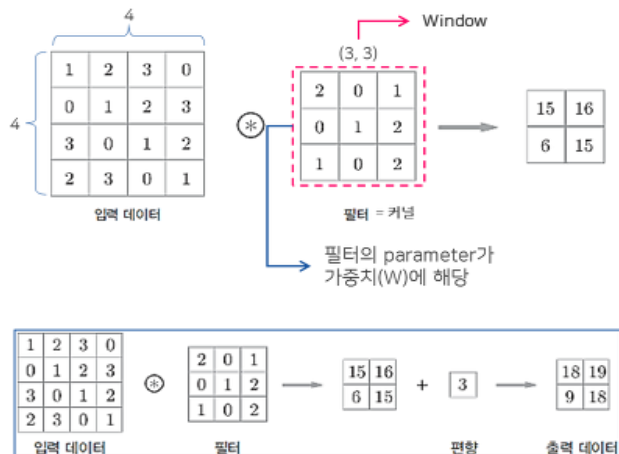
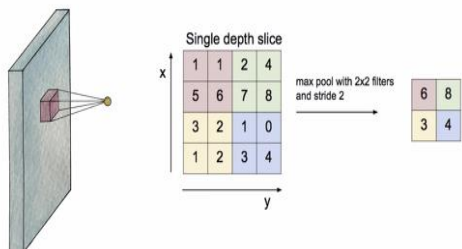
합성곱 계층 - 연산

그럼, 합성곱 계층에서 연산이 어떻게 이루어지는지 알아보도록 하자. 데이터와 필터의 모양을 (높이, 너비)로 나타내고, 윈도우(Window)라고 부른다. 여기서 입력데이터는 (4, 4), 필터는 (3, 3)이고, 필터가 바로 Conv Layer의 가중치에 해당한다.

합성곱 연산은 필터의 윈도우를 일정한 간격으로 이동해가며 계산한다. 아래 그림을 통해 계산 방법을 알아보면 합성곱 연산은 입력데이터와 필터 간에 서로 대응하는 원소끼리 곱한 후 총합을 구하게 되며, 이것을 Fused Multiply-Add(FMA)라고 한다.

마지막으로 편향은 필터를 적용한 후에 더해지게 된다.

Convolution layer and max pooling

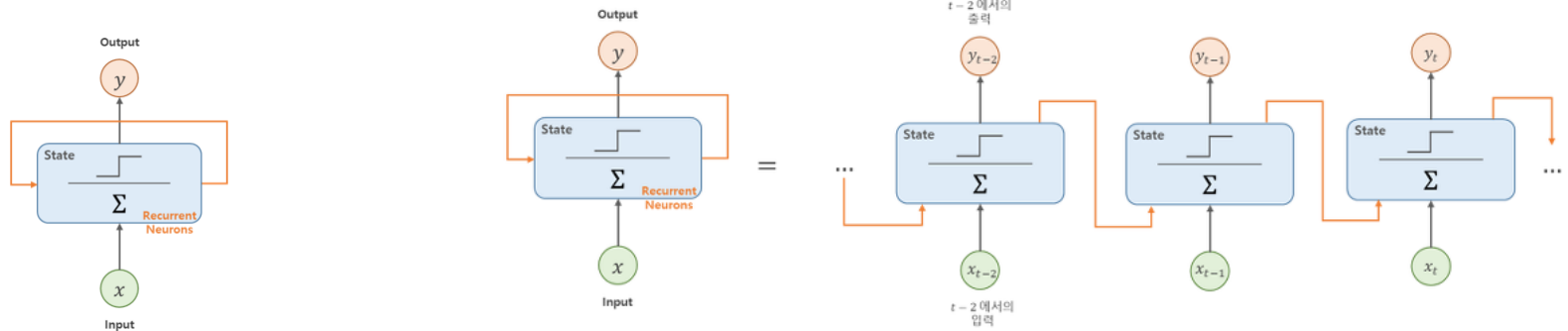


RNN(Recurrent Neural Network)

RNN(순환 신경망)은 피드포워드 신경망과 비슷하지만, 그림과 같이 출력이 다시 입력으로 받는 부분이 있다.

RNN은 입력()을 받아 출력()을 만들고, 이 출력을 다시 입력으로 받는다.

일반적으로 RNN을 그림으로 나타낼 때는 각 타임 스텝(time step) 마다 순환 뉴런을 펼쳐서 타임스텝 별 입력()과 출력()을 나타낸다.



입력과 출력 시퀀스

다양한 입력 시퀀스(sequence)를 받아 출력 시퀀스를 만들 수 있다.

